

NETRA

Network-wide Trajectory & Recognition Analytics

How the whole system works — explained from zero.

No prior knowledge assumed. Every technical word is defined the first time it appears.

SIH — PROBLEM 127

MULTI-CAMERA ANPR

Cameras today are islands

Camera 1

Sees a white bus.
Writes it down.

Camera 2

Sees a white bus.
Writes it down.

Camera 3

Sees a white bus.
Writes it down.

Nobody knows it was the SAME bus.

Because of that, two things go wrong every day:

- To follow one vehicle across a city, a police officer sits and watches footage by hand — for hours.
- Traffic control decides a road is jammed by counting cars. That is a bad measure: a wide road with many cars can still be moving perfectly fine.

We connect the cameras to each other

1

WATCH

Read every camera at the same time. Find every vehicle. Read the number plates.

2

CONNECT

Work out which sightings on DIFFERENT cameras are the same physical vehicle.

3

UNDERSTAND

Turn those links into a map trail for one vehicle, and traffic numbers for the city.

"Connect" is the real project. Watching is off-the-shelf. Connecting is the part nobody has solved — and it is where all our own code lives.

First, five words

Everything after this uses them. Learn these five and the rest of the deck is easy.

1

Model

2

Confidence

3

Frontend & Backend

4

API

5

Database

A "model" is a program that learned

Normal program

A human writes the rules by hand.

"If the shape has four wheels and is this wide, call it a car."

Works for simple things. Hopeless for photos.

A model

Nobody writes any rules.

It is shown millions of labelled photos — "this is a car", "this is a bus" — until it works the pattern out for itself.

That is how ours knows a bus.

Two words that get confused all the time:

TRAINING is showing it the millions of photos so it learns. It is slow and expensive and needs powerful computers.

RUNNING is using the already-finished model. It is fast and cheap.

We only RUN models that other people already trained and gave away free. That is exactly why this project costs zero rupees and works on a normal laptop.

Every guess comes with a "confidence"

A model never just says "car". It says "car, and I am 0.91 sure." Confidence is a number from 0 to 1.

0.95

Very sure. Trust it.

0.60

Probably right. Worth keeping, but check it against other evidence.

0.20

Almost certainly wrong. Throw it away.

Our whole project is built on NOT throwing that number away.

Most systems keep the answer and bin the confidence. We carry it all the way to the screen, so the dashboard can tell you how sure it is — and can refuse to answer when it is not sure enough.

Frontend and backend are two separate programs

FRONTEND

The part you look at.

Runs inside your web browser. Buttons, the map, the charts, the camera pictures.

It is good at showing things and bad at heavy work.

BACKEND

The part that does the real work.

Has no screen at all. Watches video, runs the AI, does the maths, saves data.

It is good at heavy work and cannot show you anything.

They are genuinely two different programs, even though both run on your one laptop. They talk to each other over the network — just like a website talks to a server, except here the "server" is also your own machine.

Ours: `backend = Python` . `frontend = JavaScript running in the browser`

That is why starting the project needs two commands in two terminal windows, not one.

An API is a doorway for asking questions

The frontend cannot reach into the backend and grab things. It has to ask, politely, through a fixed set of doors. Each door is called an "endpoint", and it is just a web address that returns data instead of a web page.

Browser asks:

`/api/stats`



Backend replies:

```
{ "vehicles": 102, "cameras_live": 3, "plates_read": 66 }
```

Our backend has 15 such doors. The browser never gets to look inside the database itself — it can only ask through these. That is what keeps the two halves independent.

The database is one file on your disk

A database is an organised place to keep information so you can find it again quickly. Think of a very strict spreadsheet that never lets you type the wrong kind of thing into a column.

Ours is SQLite

`data/netra.db`

One single file. No server to install, no username, no password.
Copy that one file and you have copied everything the system has ever seen.

Why this matters here

Everything in our system meets in this one file. The cameras only write to it. The analysis only reads and writes. The API only reads.

So any one part can be restarted without breaking the others — which matters a lot during a live demo.

It holds 8 tables. A "table" is one kind of thing, one row per item:

`cameras`

`sightings`

`plate_reads`

`vehicle_identities`

`identity_links`

`journeys`

`journey_hops`

`alerts`

Part 2

The big picture

What actually runs, and in what order.

15 API doors

8 database tables

29 Python files

3 dashboard screens

PART 2 — THE FLOW

One picture of the whole system

Video

a camera or an mp4 file

Find vehicles

YOLO draws a box round each one

Follow them

ByteTrack keeps an ID on each

Read plates

many times, not once

Vote

all the readings agree on one plate

Save

one row in the database

LIVE PASS — runs while the video plays

Match across cameras

plate + look + physics

Build journeys

route, legs, speeds

Work out traffic

congestion, origin to destination

Serve it

the 15 API doors

Draw it

the dashboard in your browser

THINKING PASS — runs afterwards, on command

This split into two passes is the single most important thing to understand about our system.

Four programs, not one

People assume a project is one thing that starts. Ours is four. Knowing which is which saves you on stage.

1

The camera workers

`make run`

Watches the videos. One separate program per camera, so if one camera breaks the others carry on. Writes what it sees to the database.

2

The thinking pass

`make resolve`
`make analytics`

Reads all the sightings, decides which ones are the same vehicle, builds journeys, works out the traffic numbers. Writes the answers back.

3

The API server

`make api`

Sits waiting on port 8000. Answers questions from the dashboard by reading the database.

4

The dashboard

`make dash`

Serves the web page on port 5173. This is what you open in your browser.

Programs 3 and 4 each need their own terminal window and stay running. That surprises people the first time.

Why each camera gets its own program

A judge will ask this. Here is the answer in two halves.

The technical reason

Python has a built-in limit that stops one Python program from truly using several CPU cores at the same time.

Giving each camera its own program steps around that limit completely. Three cameras really do run on three cores.

The real-world reason

It matches how you would actually deploy this: one small cheap computer sitting at each junction, doing its own camera.

And if one camera's video is corrupt and its program crashes, the other two never notice.

Adding a camera to the city

You add a block of text to `configs/cameras.yaml` — its name, where it is, which video to read. That is all. No code is written, ever. The supervisor reads that file and starts one program per camera listed in it.

What is in each folder

configs/

Every setting. No code at all.

Model names, thresholds, camera list, road map, watchlist. Settings live here so changing behaviour never means changing code.

data/

Everything downloaded or produced.

The three videos, the AI model files, the live preview pictures, cached map images, and netra.db — the database.

backend/

All the Python. The thinking half.

Split into core, vision, pipeline, store, identity, trajectory, analytics, api. 29 files, about 6,400 lines.

frontend/

All the dashboard. The looking half.

One brain file (App.jsx), one file that talks to the backend (api.js), and nine components — one per box you see on screen.

docs/

Your presentation material.

The deck, the PDFs, the architecture diagrams, screenshots of the running system.

Rule we followed everywhere: settings in configs/, code in backend/, nothing important hidden anywhere else.

Part 4 — the live pass

Step by step, from pixels to plates

Nine steps happen to every single vehicle, in this order, inside one camera.

1 Read the video

2 Find vehicles

3 Follow them

4 Count crossings

5 Read the plate

6 Vote on it

7 Fix the shape

8 Announce it

9 Save it

Read the video, one frame at a time

`video_source.py`

A video is not really a video. It is a fast sequence of still pictures. Each still picture is called a frame. This file opens the source and hands out frames one by one.

It handles three sources identically

- an mp4 file on disk (our demo)
- a real CCTV camera over the network
- a webcam plugged into the laptop

The rest of the code never knows which.

It also skips frames on purpose

A camera gives 25-30 frames a second. We only process 12.

A vehicle does not move much in a fortieth of a second, so the extra frames add nothing and cost twice the work.

The clever bit: one shared clock

Our three clips were filmed separately, so on their own they all start at "zero seconds". That is useless — we could never say a vehicle passed Camera 1 before Camera 2.

So each camera has an offset in its settings: 0 seconds, 240 seconds, 640 seconds. Every frame gets stamped with the shared time. Now three separate recordings behave like three real cameras on one city clock.

Find the vehicles — this is YOLO

detector.py

YOLO = "You Only Look Once"

You give it one picture. It gives you back boxes. "Car here, 0.91 sure. Bus here, 0.87 sure."

Why is it called that? Older detectors chopped a picture into hundreds of overlapping patches and asked "is there a car in THIS patch?" over and over. Slow. YOLO looks at the whole picture a single time and produces every box at once. That is the entire reason it is fast enough for video.

Which one we use

yolo11n — the "nano" size. The smallest and fastest of the family.

What it already knows

car, bus, truck, motorcycle, bicycle. It learned these from a big public photo collection.

What we changed

Nothing. We did not train it. We run it as it came, free.

Honest limitation: the model was never taught what an auto-rickshaw is. There is no such category in the photo collection it learned from, so autos get mistaken for a car or truck, or missed.

Follow each vehicle — this is ByteTrack

tracker.py

The problem: YOLO has no memory at all.

Frame 1: "a car on the left." Frame 2: "a car slightly to the right." YOLO does not know that is the same car. It has never seen frame 1. It sees each picture completely fresh.

ByteTrack gives it a memory.

It predicts where each box should move to next, matches that prediction against the new frame, and keeps a number stuck to the vehicle. Car #14 stays car #14 the whole time it is on screen.

Why we picked this one over others:

Most trackers throw away the boxes YOLO was unsure about. But when a vehicle goes half-behind a bus, its confidence drops — so those trackers lose it exactly when it gets hard. ByteTrack keeps the weak boxes and gives them a second chance at matching. It holds on through partial blocking. That is the whole idea, and it is why it is the standard choice.

The moment a vehicle leaves the frame, its ID dies forever.

What ByteTrack can do

Keep an ID on a vehicle inside ONE camera's view. Nothing more. It was never designed to do more.

What we built ourselves

Everything that happens after that ID dies. Deciding that vehicle #14 on Camera 1 and vehicle #7 on Camera 3 are the same real vehicle.

Count what crosses the line

`counting.py`

We draw an invisible line across the road in each camera. When a tracked vehicle crosses it, we add one to a counter — and we note which way it was going.

CAM01

inbound / outbound

CAM02

eastbound / westbound

CAM03

entry / exit

The small detail that makes it robust

We do not store the line as "pixel 340". We store it as a fraction of the picture — 0.62 means "62% of the way down".

So if you swap in a camera with a different picture size, the line stays in the right place on the road. Nothing needs re-drawing. Each camera also names its own two directions, because "inbound" means something different at a junction than at an airport gate.

Read the number plate

`anpr.py`

OCR

Optical Character Recognition. Turning a PICTURE of text into real text. Same idea as your phone copying words out of a photo.

ANPR

Automatic Number Plate Recognition. OCR aimed specifically at number plates.

It is two steps, not one — people always assume one:

A

Find the plate

One model looks at the vehicle and draws a box round the plate rectangle.

B

Read the plate

A second model looks inside that box and reads the characters.

The property we actually needed

Our reader does not just say "KA01AB1234, 0.78 sure". It gives a confidence for EVERY SINGLE CHARACTER — how sure about the K, then the A, then the 0. Nothing on the next slide is possible without that.

We never trust a single frame. We vote.

`consensus.py`

A vehicle is on screen for maybe 60 frames. We read the plate about 15 times. Most readings differ slightly.

What came back

KA01AB1234	frame 12
KA01A81234	frame 15
KA0IAB1234	frame 18
KA01AB1284	frame 21
KA01AB1234	frame 24

What we do with them

We hold a vote — one character position at a time.

And each vote is WEIGHTED by how confident the reader was about that character. A "0" read at 0.95 confidence out-votes an "O" read at 0.40 confidence.

The winning characters form the final plate.

Result: KA01AB1234

One agreed answer from fifteen imperfect ones. This is the sentence to memorise: "We never trust a single frame — we vote across every frame we got, weighted by confidence."

Indian plates have a shape. We use it.

plate_grammar.py

KA 01 AB 1234

two letters
the state

1-2 digits
the district

1-3 letters
the series

This is a fixed, legally defined shape. Any reading that does not fit it is wrong, and we can often tell exactly where.

Repair

Position 3 must be a letter. The reader gave us a "0" there. In that position a "0" is almost certainly an "O", so we correct it.

Fair comparison

When comparing two plates we know which mistakes cameras really make. Those cost a quarter of a point. A random mismatch costs a full point.

The character pairs cameras genuinely confuse:

0 ↔ O

8 ↔ B

1 ↔ I

5 ↔ S

2 ↔ Z

D ↔ 0

So "KA01AB1234" and "KA0IAB1234" are treated as a near-miss of the same vehicle, not two different vehicles.

Announce it on a noticeboard

`core/bus.py`

When a vehicle finally leaves the frame, its sighting is finished. The camera program does NOT save it to the database itself. Instead it pins a note to a noticeboard: "sighting completed, here are the details."

That sounds like extra work

It is one extra step. A separate listener reads the noticeboard and writes to the database.

So why bother?

Because it is our scaling story

The camera program has no idea a database even exists. It just announces.

Swap the in-memory noticeboard for Kafka — the industrial version used by companies handling millions of events a second — and you run 500 cameras.

And nothing else in the codebase changes.

Not the camera code. Not the database code. Not the listener — its instructions are written to work identically whether the note came from memory or from Kafka.

That is 106 lines of code that turn "it could scale" from a claim on a slide into something you can point at in the repository.

Save it — and the live pass is done

`camera_worker.py`

finishes the sighting and announces it



`bus.py`

the noticeboard carries the note



`subscriber.py`

listens, picks it up



`repository.py`

the only file allowed to write to the database



`data/netra.db`

one new row

One rule we kept: no other file in the whole project writes to the database. If you want to know how something is stored, there is exactly one place to look.

Part 5

The heart of the project

Everything so far exists inside one camera.

Now we cross between cameras — and this is the part that is genuinely ours.

`identity/resolver.py`

The one question the system has to answer

**Camera 1 saw a vehicle at 10:02. Camera 3 saw a vehicle at 10:09.
Same vehicle, or two different vehicles?**

We use three signals. They are not equal — and the difference between them is the whole design.

1 The plate

The main evidence

2 The look

Backup evidence

3 Physics

A veto — it can say NO

Most systems only ever match. Ours can also refuse — and it shows you what it refused.

Signal 1: the plate. Signal 2: the look.

1 The plate

We compare the two voted plates using the confusion-aware scoring from step 7 — so a camera's honest mistakes do not count as a different vehicle.

This is the strongest signal we have. A full plate match is very hard to get by accident.

2 The look

The rough colour and shape of the vehicle body.

This is deliberately WEAK evidence. Thousands of white cars exist. It can support a plate match, but on its own it proves nothing — and we treat it that way.

An important honesty setting

In our configuration we require a plate before we will claim two sightings are the same vehicle. Appearance alone is never enough.

We found this the hard way. In an early run, the appearance score sat higher than our accept threshold, and the system happily merged nine unrelated sightings into one "vehicle". We lowered the plateless limit and made a plate compulsory. That bug is why the rule exists.

Signal 3: physics

Plate and look are evidence FOR a match. Physics is different — it is a constraint that can overrule both.

The example to use on stage

Two sightings, plates match perfectly

score 1.00

Distance between the two cameras

4 kilometres

Time between the two sightings

30 seconds

Speed that would require

480 km/h

REJECTED. No car does 480 km/h. It cannot have been the same vehicle, however well the plates matched.

Where do the distances come from? `configs/city_graph.yaml` — our map of which cameras connect to which, how far apart they are, and the normal speed of each road.

Turn confirmed links into a journey

`trajectory/engine.py`

Once we know which sightings belong together, we string them into one journey and work out everything measurable about it.

Route

CAM01 → CAM02 → CAM03

Each leg

when it left, when it arrived, how long, how far, how fast

Totals

total distance, total time, average speed

Confidence

the score of the WEAKEST link in the chain — a journey is only as trustworthy as its shakiest step

Coverage gaps — a free bonus we did not plan for

If the road map says a vehicle MUST have driven past Camera 2, but Camera 2 recorded nothing, we flag it. That is a broken or badly-aimed camera, found automatically. It is a camera fault, not a traffic one — and we say so on screen.

Congestion from time taken, not cars counted

[analytics/metrics.py](#)

The usual way

Count the cars. Many cars = jam.

But a six-lane road with many cars can be moving perfectly. And an empty road blocked by one broken-down lorry is a total jam. Counting gets both of these wrong.

Our way

We already know exactly how long real vehicles took to get from Camera 1 to Camera 2. We compare that against how long the road normally takes.

Taking twice as long as usual IS congestion, by definition.

Where "normally" comes from

Not from a guess. We take all the trips we have actually measured on that road and use the fastest 15% of them as "free flow". The road tells us its own normal speed.

Also produced here: which camera pairs vehicles travel between most (origin to destination), and how healthy each camera is — how many vehicles it saw versus how many plates it managed to read.

Watchlist alerts — worded carefully on purpose

`analytics/alerts.py` · `configs/watchlist.yaml`

You put plates of interest in a settings file. When one is seen, the dashboard raises an alert.

"possible watchlist match 01AB12334 at CAM01"

"Plate similarity 1.00. Flagged for human review, not confirmed."

— *the actual text our system prints*

Read those words again. Every one of them is deliberate.

- "possible" — because a camera reading is evidence, not proof.
- "flagged for human review" — because a person decides, not the software.
- "not confirmed" — because we never let the screen imply more certainty than we have.
- This is a system that may one day be pointed at real people. The wording is a design decision, not decoration.

Part 7

Getting it onto a screen

All the answers now sit in a database file. Nobody can see a database file. This part turns it into something a human can look at.

The 15 doors, and who knocks on them

api/main.py — built with FastAPI, a tool that turns Python functions into web addresses

DOOR	WHAT IT RETURNS	WHAT USES IT ON SCREEN
/api/health	is the backend alive?	the green dot, top right
/api/cameras	list of cameras + where they are	the map, the camera boxes
/api/stats	the six big numbers	the number bar at the top
/api/sightings	recent vehicles seen	the Live Detections list
/api/journeys	rebuilt journeys	the Trace screen list
/api/analytics	all the traffic numbers	the Analytics screen
/api/alerts	watchlist hits	the red Alerts panel
/api/rejected-links	matches we REFUSED	"Links the system refused"
/api/live/CAM01	the moving camera picture	the three video boxes
/ws/events	a line held open to push new events instantly	the live feed

Five more exist for plates, identities, congestion and map tiles.

The dashboard is built from small pieces

React is a tool for building a screen out of reusable blocks called components. Each visible box on our dashboard is one file.

App.jsx

The brain. Asks the backend for everything, remembers the answers, decides which screen shows.

CityMap.jsx

The map, the camera dots, the coloured route lines.

EventFeed.jsx

The scrolling list of vehicles just seen.

PlateSearch.jsx

The search box and the list of journeys.

AnalyticsView.jsx

The graph and the traffic tables.

api.js

The only file that knows how to talk to the backend. Every other file asks this one.

CameraGrid.jsx

The three live camera pictures.

KpiBar.jsx

The six big numbers along the top.

JourneyTimeline.jsx

One vehicle's trip, leg by leg.

AlertsPanel.jsx

The red watchlist alerts.

App.jsx re-asks the backend every 2 seconds for fast things and every 8 seconds for slow ones — that is why the numbers on screen keep climbing while you watch.

How the browser finds the backend

The dashboard lives on port 5173. The backend lives on port 8000. A "port" is just a numbered door on your computer, so two programs can both use the network without colliding.

But the dashboard code asks for `/api/stats` — with no port number in it at all. So how does it arrive?

`vite.config.js` quietly forwards anything starting with `/api` over to port 8000.

`browser asks 5173/api/stats` → `forwarded to` → `backend at 8000/api/stats`

The dashboard code never has to know the backend's address.

No CORS problems

Browsers block a page on one address from calling another, for security. Because everything appears to come from one address, that rule never fires.

Nothing to change later

When you eventually serve both from one real address in production, the dashboard code is already correct. Not one line changes.

The "live video" is not video at all

Streaming real video into a web page is genuinely hard and breaks in interesting ways. We refused to do it.

1

Each camera program writes one JPEG picture to disk

`data/live/CAM01.jpg` – overwritten again and again

2

The backend serves those pictures as a rapid sequence

`a format every browser has understood for 20 years`

3

The page shows it in an ordinary image tag

`no player, no plugin, nothing to go wrong`

Boring on purpose. If the picture feed dies, the numbers, the map and the journeys all carry on working. Nothing takes the dashboard down with it.

Part 8

Follow one bus all the way through

Every file, in order, for one real vehicle.

If you can tell this story, you understand the project.

A bus enters CAM01 at 10:02

`video_source.py`

Frame arrives. Stamped 10:02:14 on the shared city clock.

`detector.py`

YOLO: "bus, 0.88 sure, this box."

`tracker.py`

ByteTrack: "this is vehicle #34." Same number for the next 60 frames.

`counting.py`

It crosses the line. inbound count goes up by one.

`anpr.py`

Plate read on every 3rd frame. 15 readings, slightly different.

`consensus.py`

Weighted vote across all 15 → "KA01AB1234".

`plate_grammar.py`

Fits the Indian plate shape. No repair needed. Accepted.

`bus.py`

Bus leaves the frame. Sighting finished and announced.

`subscriber.py` → `repository.py`

One new row in the sightings table.

`resolver.py`

Later: same plate at CAM02 at 10:06. 1.6 km in 4 min = 24 km/h. Sensible. LINKED.

`engine.py`

Journey built: CAM01 → CAM02 → CAM03, 4.7 km, 10m 34s, confidence 0.84.

`api/main.py`

The dashboard asks /api/journeys. This row comes back.

`App.jsx` → `CityMap.jsx`

The route is drawn on the map. You see the bus's trip.

The eight tables

store/models.py

cameras	One row per camera. Name, where it is, which way it faces.
sightings	One row per vehicle seen by one camera. The main table.
plate_reads	Every individual plate reading, before the vote. Kept as proof.
vehicle_identities	One row per REAL vehicle, after matching across cameras.
identity_links	One row per decision: these two sightings are the same vehicle.
journeys	One row per rebuilt trip across the city.
journey_hops	One row per leg of a trip. CAM01 to CAM02 is one hop.
alerts	One row per watchlist hit.

Notice we keep `plate_reads` — every raw reading, not just the voted answer. That is deliberate: it means we can always show our working.

What you can safely change on the day

SAFE TO CHANGE

`detection.imgsz`

Lower it to 480 if the laptop is slow. Faster, slightly less accurate.

`target_fps`

Frames processed per second. Lower = faster.

`anpr.every_n_frames`

Read plates less often to save time.

`runtime.mode`

"realtime" looks live. "fast" processes as quickly as it can.

`watchlist.yaml`

Add or remove plates to alert on.

DO NOT TOUCH BEFORE A DEMO

`clock_offset_s`

The camera time offsets. These set the implied travel speeds. Change one and the physics check will correctly start rejecting real matches.

`city_graph.yaml` `distances`

Same reason. The whole veto depends on these being honest.

`accept_threshold`

How sure we must be to link two sightings. Loosen it and the system starts inventing matches.

`detection.classes`

Which vehicle types count.

Golden rule: every tunable number lives in configs/, never inside the code. That is why "can it do 50 cameras?" has a one-line answer.

From a fresh terminal, in order

- 1** `make setup` Builds a private Python folder for this project and downloads the libraries into it. Needs internet. Once per machine.
- 2** `make dash-setup` Downloads the dashboard libraries. Needs internet. Once per machine.
- 3** `make warm` Downloads the AI model files. Needs internet. Once. NEVER skip this — otherwise the first run tries to download mid-demo.
- 4** `make run` Starts the camera programs. Watches the videos, fills the database. Takes a couple of minutes.
- 5** `make resolve` The matching pass. Decides which sightings are the same vehicle.
- 6** `make analytics` Works out the traffic numbers.
- 7** `make api` Starts the backend on port 8000. NEEDS ITS OWN TERMINAL. Leave it running.
- 8** `make dash` Starts the dashboard on port 5173. NEEDS A SECOND TERMINAL. Leave it running.

Then open `http://localhost:5173` in your browser. Steps 1-3 are needed only once, ever.

The four things that actually go wrong

Dashboard shows nothing at all

The backend is not running. Check terminal 2. Then open localhost:8000/api/health — it should say ok.

Numbers are all zero

You never ran the camera pass, or the database is empty. Run make run, then make resolve, then make analytics.

No journeys on the Trace screen

make resolve has not been run since the last make run. Run it.

Install fails with a network error

No internet, or the wrong Python version. torch needs Python 3.11 or 3.12, not 3.13. Rebuild with: make setup PY=python3.12

Known bug: do NOT type a plate into the search box and press Trace — that door returns an error. Click a journey in the list instead.

Part 13

Facing the judges

Answers built only from what the code actually does.
Nothing here is invented.

MEMORISE THIS ONE

The 30-second answer

"A city has hundreds of cameras that do not talk to each other. Camera 1 saw a vehicle, Camera 5 saw a vehicle, and nothing today can tell you it was the same one.

We find every vehicle, read its plate many times over and vote on the answer, then work out which sightings on different cameras are the same physical vehicle — using the plate, the appearance, and physics, where physics can veto a match that would need an impossible speed.

From those confirmed links we rebuild each vehicle's journey across the city, and measure congestion from real travel times instead of car counts.

Everything is open source. It runs on a laptop. It costs nothing."

Three phrases to land in that answer:

"vote, not one frame"

"physics can veto"

"zero cost, runs on a laptop"

Questions you will definitely get

Why YOLO?

It looks at a whole frame once instead of scanning it in patches, which is what makes it fast enough for video. We use the smallest version and did not train it — the model name is a setting, so we can swap it without touching code.

How do you track a vehicle?

ByteTrack keeps an ID on it inside one camera. It keeps low-confidence boxes that other trackers throw away, so it holds on when a vehicle goes half-behind a bus. But that ID dies the moment the vehicle leaves the frame.

What if OCR reads the plate wrong?

We assume it will. We read the plate on many frames and vote character by character, weighted by confidence. Then we check the result against the legal Indian plate shape and repair it. And when we compare two plates we know which characters cameras genuinely confuse.

What is actually AI here?

Two learned models — one finds vehicles, one reads plates. Everything else, including the matching that makes this project interesting, is our own engineering. We say that plainly rather than calling the whole thing AI.

Questions designed to catch you out

How do you know it is the same vehicle across cameras?

Three signals. Plate similarity is the evidence. Appearance corroborates. Travel time is a constraint that can veto — if the link needs 480 km/h we reject it however well the plates matched. In our settings a plate is compulsory; appearance alone is never enough.

How does this scale to thousands of cameras?

Cameras never call each other. Each publishes events to a bus. Today that bus runs in memory; swapping in Kafka is one new class behind an interface we already wrote. Nothing else changes. And a new camera is a block of settings, not code.

What if a camera goes offline?

Each camera is its own program, so the others carry on. Better than that — if the road map says a vehicle must have passed a camera that recorded nothing, we flag it as a coverage gap. We find the dead camera automatically.

What about poor lighting or bad plates?

Honest answer: a plate needs to be roughly 80 pixels wide to be readable, and most public CCTV is mounted too high and wide for that. That is a camera-placement problem, not a software one — and it is exactly why our system reports confidence instead of pretending.

What our system does NOT do

Auto-rickshaws are not a category

The detector was never taught what one is. They get absorbed into car or truck, or missed. Fixing it means training on an Indian vehicle dataset.

Accuracy is not yet measured on real footage

Our numbers come from our own test clips. We have the evaluation script ready; we need real filmed footage with a known pass order to fill it in.

Plate search has a live bug

Typing a plate and pressing Trace returns an error. Clicking a journey in the list works. We know exactly which three lines cause it.

We did not train any models

Both models are free, open source, and used as-is. Our contribution is everything built around them.

Saying these first is a strength. A judge who finds a hidden weakness stops trusting everything else you said.

KEEP THIS OPEN DURING THE HACKATHON

Cheat sheet

THE STACK

Detection	YOLO (yolo11n)
Tracking	ByteTrack
Plates	fast-alpr
Backend	Python + FastAPI
Database	SQLite, one file
Frontend	React + Vite
Map	Leaflet
Charts	Recharts
Cost	zero, no API keys

THE PIPELINE

```
frame
  → find vehicles
  → follow them
  → read plate x15
  → vote on it
  → fix the shape
  → save
  → match across cameras
  → build journey
```

FILES THAT MATTER

resolver.py	the heart
camera_worker.py	the live loop
consensus.py	the vote
plate_grammar.py	plate rules
engine.py	journeys
models.py	the 8 tables
api/main.py	the 15 doors
App.jsx	the dashboard

IF A JUDGE SAYS "HOW DOES IT WORK?"

"YOLO finds the vehicles. ByteTrack follows each one inside a single camera. We read the plate on many frames and vote, instead of trusting one. Then our resolver decides whether the vehicle at Camera 1 and the vehicle at Camera 3 are the same one — using the plate, the appearance, and physics, where physics can veto a match that would need an impossible speed. From the confirmed links we rebuild the journey and measure congestion from real travel times. All open source, runs on a laptop, zero cost."

**The one line
to remember**

**Most systems only match.
Ours can also refuse —
and it shows you what it refused.**